

Programmierung in C++

Organisatorisches

Prof. Dr. Bernd Lüdemann-Ravit

Team



Prof. Bernd Lüdemann-Ravit

Sprechstunde

(auf Anfrage per Mail):

bernd.luedemann-ravit@hs-kempten.de



Dan Eisenkrämer (B.Sc.)

dan.eisenkraemer@hs-kempten.de

Discord-Tag:

todesengel0



Vinzenz Funk (B.Sc.)

vinzenz.funk@stud.hs-kempten.de

Discord-Tag:

future1337

Gerne per Du!

Moodle



- Alle Vorlesungen und Übungen sowie weitere Informationen im Moodle Kurs
- Kursname: **Programmieren 2 mit C++**
- Einschreibeschlüssel: **Bjarne**



Vorlesung & Übungen

Termine und Themen



KW	Termin VL	Inhalt Vorlesung	Ausgabe Übungsblatt Mo	Übung Mo	Übung Di	Übung Mi	Übung Do	Übung Fr
12	17.03.	Organisatorisches + Überblick Grundlagen	0					
13	24.03.	Basic Objektorientierung	1	0	0	0	0	0
14	31.03.	Basic Objektorientierung + Kopieren & Verschieben	2	1	1	1	1	<i>Ostern</i>
15	07.04	Kopieren & Verschieben	3	<i>Ostern</i>	2	2	2	1
16	14.04	Vererbung	4	2	3	3	3	2
17	21.04.	Klassenbeziehungen	5	3	4	4	4	3
18	28.04.	Speicherverwaltung	6	4	5	5	5	<i>T.d.Arbeit</i>
19	05.05.	Speicherverwaltung + Container	7	5	<i>H.-Messe</i>	<i>H.-Messe</i>	6	4+5
20	12.05.	Templates	8	6	6	6	7	6
21	19.05.	Funktionale Programmierung + Programmierkonz.	9	7	7	7	8	7
22	26.05.	<i>Pfingsten (Vorlesungsfrei)</i>		<i>Pfingsten</i>	<i>Pfingsten</i>	<i>Pfingsten</i>	<i>Pfingsten</i>	<i>Pfingsten</i>
23	02.06.	Programmierkonzepte	10	8+9	8+9	8+9	<i>Fronleich.</i>	8+9
24	09.06.	Patterns + Fehlerbehandlung	11	10	10	10	9+10	10
25	16.06.	Fehlerbehandlung + Multithreading	12	11	11	11	11	11
26	23.06.	Multithreading + Input & Output	13	12	12	12	12	12
27	30.06.	Input & Output + Besprechung Evaluation	Probeklausur	13	13	13	13	13
28	07.07.	Prüfungsvorbereitung + Vorstellung IPI		Fragen	Fragen	Fragen	Fragen	Fragen



- Die Vorlesung behandelt die **Grundlagen der Programmiersprache C++**
 - Euch soll ein **Überblick** über verschiedene Themen und Programmiertechniken gegeben werden, die Ihr in den Übungen vertiefen werdet.
 - Die Vorlesung ist so aufgebaut, dass wir uns für die Themen die Zeit nehmen können, die sie brauchen.
- Nachfragen sind jederzeit willkommen und erwünscht!
 - Gebt Bescheid, wenn Ihr eine kurze Denkpause benötigt.
 - Am Ende der Unterkapitel gibt es eine kurze Zusammenfassung und Fragerunde.
- Fehler sind menschlich und auch in dieser Vorlesung nicht unvermeidbar
 - Wenn Ihr glaubt, einen Fehler gefunden zu haben, meldet Euch per E-Mail an Bernd, Dan und Vinz! => Wer die meisten Fehler findet, bekommt eines der Bücher geschenkt!
 - Wir versuchen, die Vorlesung stets zu verbessern und euer Feedback wird dabei berücksichtigt!



*„Programmieren lernt man durch Programmieren,
was man durch Programmieren lernt.“ – Prof. Dr. Stefan Frenz*

- Die Übungsstunden sind **Übungsbesprechung** und **Betreute Programmierzeit**
- Jedes Übungsblatt besitzt Aufgaben zu den **Grundlagen**, die gemeinsam besprochen werden.
- Die weiteren Aufgaben sind praktische **Programmierübungen**, mit denen Ihr die Programmierkonzepte zu lernen beginnen könnt.
- Bei den Programmierübungen werdet Ihr durch den Übungsleiter betreut.
- Keine Abgaben/ **Keine Prüfungsvorleistung!**



- Wenn euch bei der Bearbeitung der Übungsblätter **Fragen** aufkommen, habt ihr verschiedene Optionen:
 1. In der Übungsstunde den **Übungsleiter** fragen
 2. Fragt eure **Kommilitonen** z.B. in der Übung oder auf dem **Fakultäts-Discord-Server** im Channel für Programmieren 2
 3. **Dan** und **Vinz** via Discord (gerne auch als Direktnachricht)
 4. **Bernd** per E-Mail oder nach der Vorlesung

Discord Server
Fakultät Informatik:
discord.gg/TWeBSej



Programmieren mit C++



Ratschläge

- **Keine Panik!** Mit Zeit und Übung wird alles verständlich.
 - In der Vorlesung wirst Du mit **sehr vielen Inhalten und Konzepten** konfrontiert.
 - Versuche nicht, alles sofort zu verstehen; Sondern habe **Interesse und Spaß es selbst auszuprobieren!**
- Verwende nicht nur die Kernfunktionalitäten von C. Viele Funktionen lassen sich am besten indirekt durch **Bibliotheken** verwenden.
 - **Erfinde das Rad nicht neu.**
- Du musst nicht jedes Detail von C++ kennen, um ein guter Programmierer zu werden.
 - **Programmiertechniken** (die „Programmier-Denkweise“) sind wichtiger als sprachspezifische Umsetzungen
- **Kommentiere** deine Absichten
 - Nicht das, was 1 zu 1 aus dem Code erkennbar ist
- Strebe einen **einheitlichen Programmierstil** an

Programmieren mit C++



Verwendung von LLMs (insb. Coding Co-Piloten)

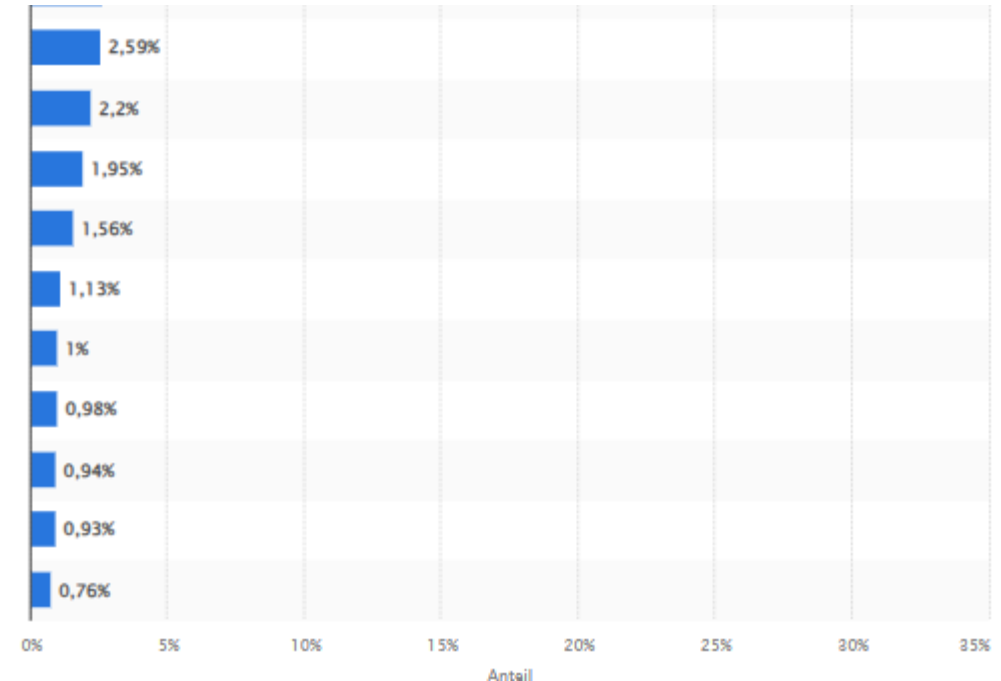
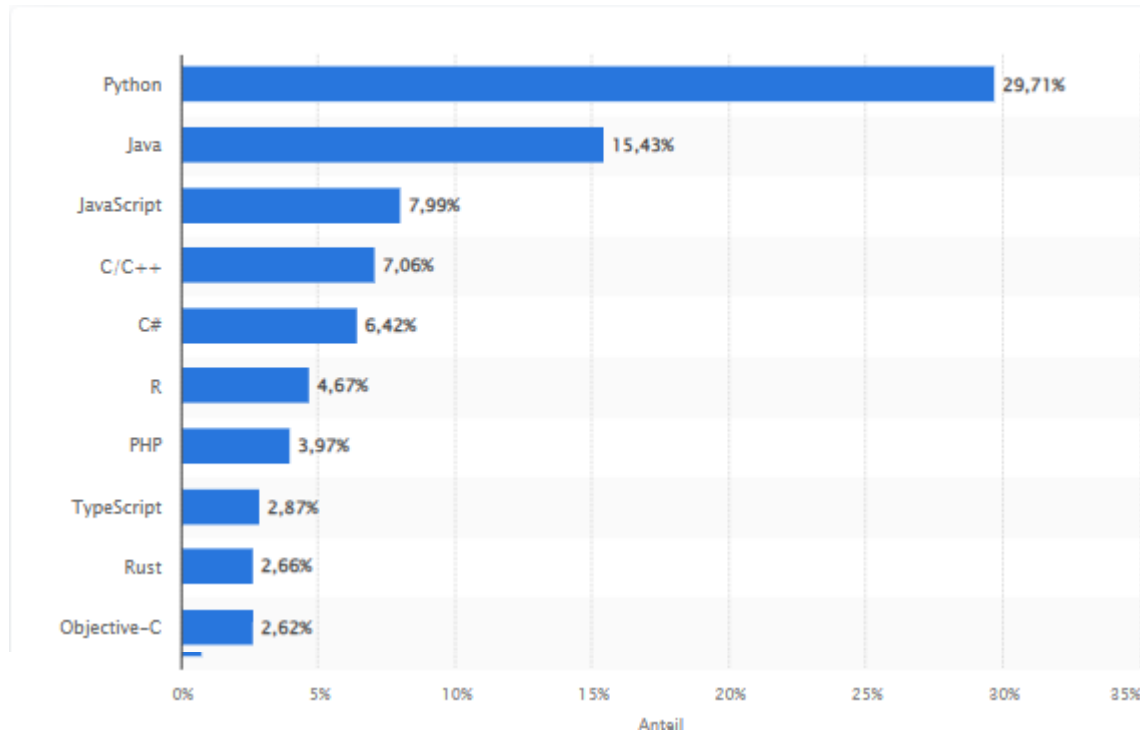
- Large Language Models und insbesondere Co-Piloten (z.B. GitHub Co-Pilot) unterstützen bei **häufig geschriebenem Code**
 - Sie **orientieren sich auch am eigenen Programmierstil**
- Wenn Du ein Thema nicht verstehst, wirst Du auch nicht verstehen, was der Co-Pilot programmiert hat und ob es sinnvoll ist
 - Beim Aufbau von Verständnis ist **selbstständiges Nachdenken und Knobeln** essenziell
 - LLMs können unterstützen, aber einem (noch) **nicht die Arbeit abnehmen**
- **Tipp:** Lernen LLMs **kritisch zu hinterfragen, zu beurteilen und zu überprüfen**
 - Zu Lernzwecken **Co-Piloten deaktivieren**, um erst einmal selbst auf die Lösung zu kommen!
 - Bevor man LLMs befragt: Im **Skript** und auf **Programmier-Webseiten** nachschauen, **Kommilitonen** und **Übungsleiter** fragen.



- Prüfungsrelevant sind **alle behandelten Vorlesungsthemen**
- Wenn ihr die in den **Übungen** vertieften Konzepte der Vorlesungen beherrscht, seid ihr bestens auf die Prüfung vorbereitet.
 - Es liegt in eurer Eigenverantwortung, dass ihr die Übungen macht: Sie werden die beste Vorbereitung auf die Prüfung sein und eine frühzeitige Bearbeitung hilft euch dabei, die darauffolgenden Vorlesungsthemen bereits besser verstehen zu können.
- Die Prüfung soll euer **grundlegendes Verständnis** und die **notwendigen Programmiertechniken** abfragen.
 - Und nicht, wie gut ihr die Softwaredokumentation auswendig könnt.
- **Zugelassene Hilfsmittel (Open-Book-Klausur):**
 - Ihr dürft alles an analogen Hilfsmitteln mitnehmen (Bücher, das Skript, Ausdrücke, eigene Zusammenfassungen, etc.)
 - Elektronische Hilfsmittel sind **nicht** erlaubt (u.a. Smart Watches, ChatGPT, mobile Telefone)

Programmiersprachen

Beliebtesten Programmiersprachen in 12/2024 (PYPL-Index)





Das aktuelle TIOBE-Ranking im Überblick (Stand: Juni 2024, Quelle: www.tiobe.com):

- **Platz 1:** Python (15,39 Prozent)
- **Platz 2:** C++ (10,03 Prozent)
- **Platz 3:** C (9,23 Prozent)
- **Platz 4:** Java (8,40 Prozent)
- **Platz 5:** C# (6,65 Prozent)
- **Platz 6:** JavaScript (3,32 Prozent)
- **Platz 7:** Go (1,93 Prozent)
- **Platz 8:** SQL (1,75 Prozent)
- **Platz 9:** VisualBasic (1,66 Prozent)
- **Platz 10:** Fortran (1,53 Prozent)
- **Platz 11:** Delphi/ObjectPascal (1,52 Prozent)
- **Platz 12:** Swift (1,27 Prozent)
- **Platz 13:** Assembler (1,26 Prozent)
- **Platz 14:** MATLAB (1,26 Prozent)
- **Platz 15:** PHP (1,22 Prozent)
- **Platz 16:** Scratch (1,17 Prozent)
- **Platz 17:** Rust (1,17 Prozent)
- **Platz 18:** Ruby (1,11 Prozent)
- **Platz 19:** Kotlin (1,01 Prozent)
- **Platz 20:** COBOL (0,96 Prozent)

Programmiersprachen

Die fünf führenden Programmiersprachen auf einen Blick



Name	Komplexität	Performance	Plattform-unabhängigkeit	Garbage Collection	Objektorientierung	KI-Eignung	TIOBE-Ranking 06/2024
Java	Überschaubar	Hoch	Maximal	Ja	Ja	Mittel	8,40 %
C#	Hoch	Sehr hoch	Mittel	Nein	Nein	Mittel	6,65%
Python	Gering	Mittel bis hoch	Hoch	Ja	Ja	Hoch	15,39 %
C/C++	Hoch	Sehr hoch	Mittel	Nein	Teilweise	Hoch	9,23 % / 10,03 %
JavaScript	Überschaubar	Niedrig	Ja	Ja	Ja	Mittel	3,32%

C++: Eine kurze Einführung



Wer verwendet C++?

C++ wird in verschiedenen Bereichen verwendet, beispielsweise:

- Low-Level-Systemprogrammierung (Betriebssysteme, Gerätetreiber und eingebettete Systeme).
- Spieleentwicklung (3D-Spiele und Game-Engines).
- Desktopanwendungen (Textverarbeitung, Webbrowser und Multimediasoftware).
- Wissenschaftliches Rechnen (effiziente komplexe Berechnungen und Simulationen).
- Hochfrequenzhandel (Finanz-Anwendungen, die eine geringe Latenz für die Handelsausführung erfordern).

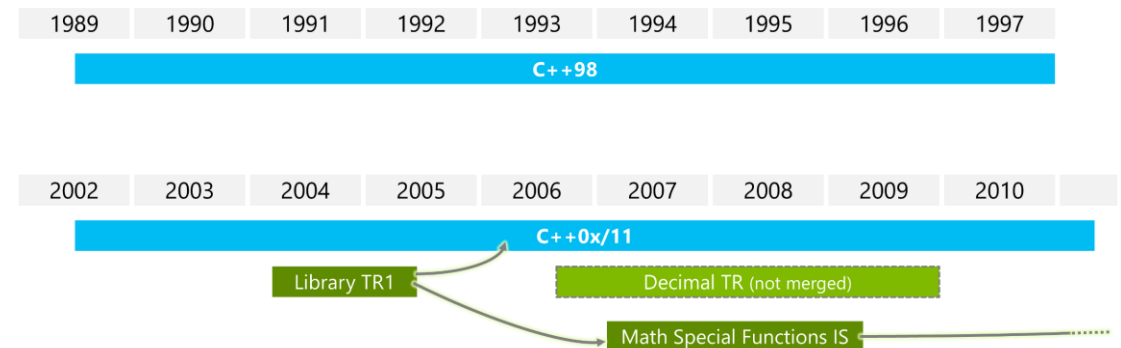
Statistiken zeigen, dass C++ unter Spieleentwicklern die beliebte Programmiersprache ist . Darüber hinaus wird C++ aufgrund ihrer Vielseitigkeit gerne von Entwicklern verwendet, insbesondere von solchen mit umfassenden Kenntnissen in Computerprogrammierung.

C++: Eine kurze Einführung

Geschichte



- C++ wurde ab 1979 von Bjarne Stroustrup bei AT&T als Erweiterung der Programmiersprache C entwickelt.
 - Damals noch unter dem Namen „C with Classes“
 - 1984 erhielt C++ seinen Namen und 1985 seine erste kommerzielle Veröffentlichung
 - 1997 wurden in die mit C++ entwickelte „Standard Bibliothek“ Funktionalitäten der „Standard Template Library“ (STL) hinzugefügt
 - Der ANSI-Standard für C++ wurde im Juni 1998 festgelegt
 - 2002 begann die Arbeit des überarbeiteten Standards C++0x
 - 2011 wird C++0x zum ISO C++11 Standard



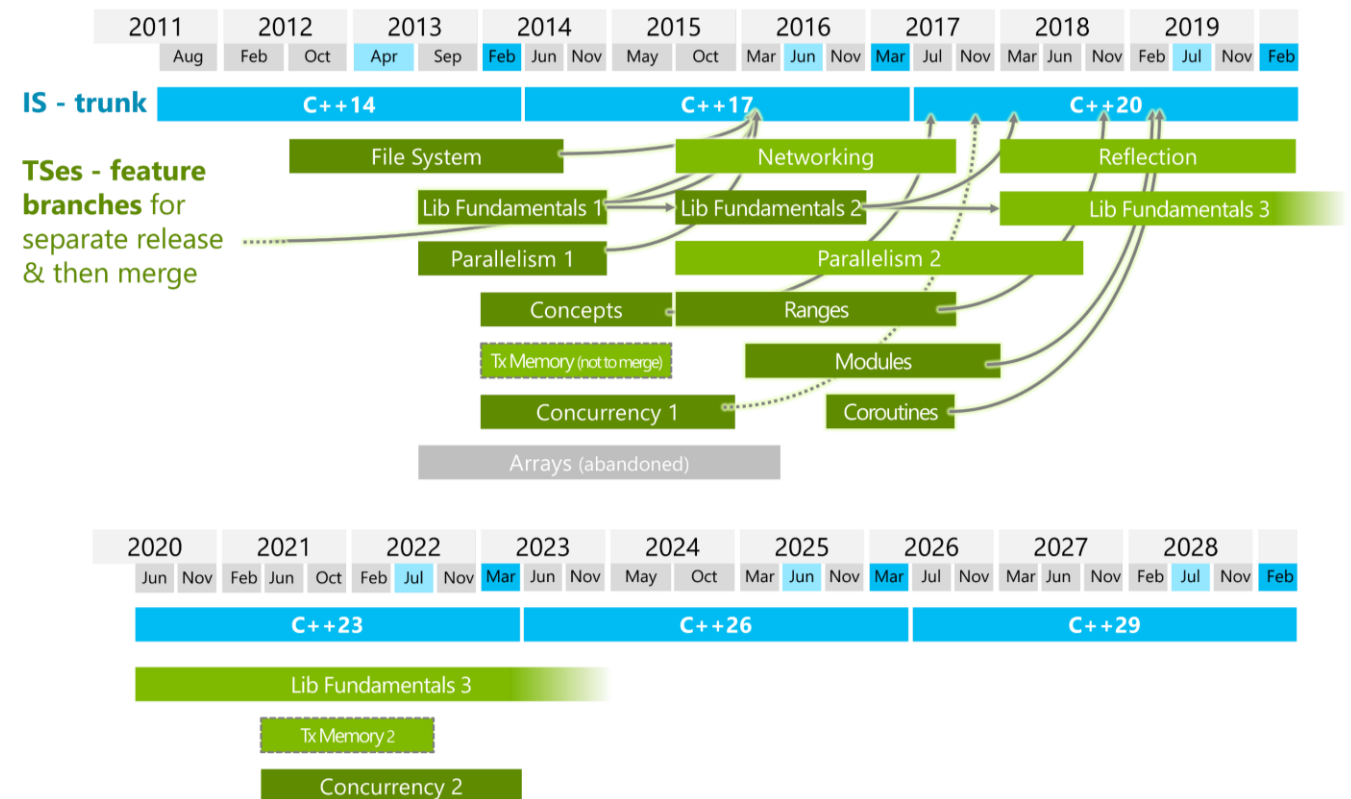
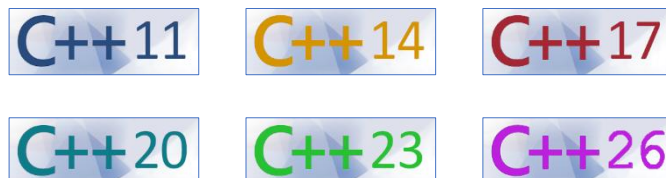
<https://isocpp.org/std/status>

C++: Eine kurze Einführung

Geschichte



- Mit Beginn der Erweiterung des C++ Standards durch C++0x wurde eine stetige Entwicklung an der Standard Bibliothek gestartet
 - In einem 3-Jahres-Rhythmus wird der ISO-Standard mit einer neuen Version erweitert
- In der Vorlesung ist für Funktionalitäten aus den Standards ab C++11 markiert, aus welchem Standard sie entstammen:



<https://isocpp.org/std/status>

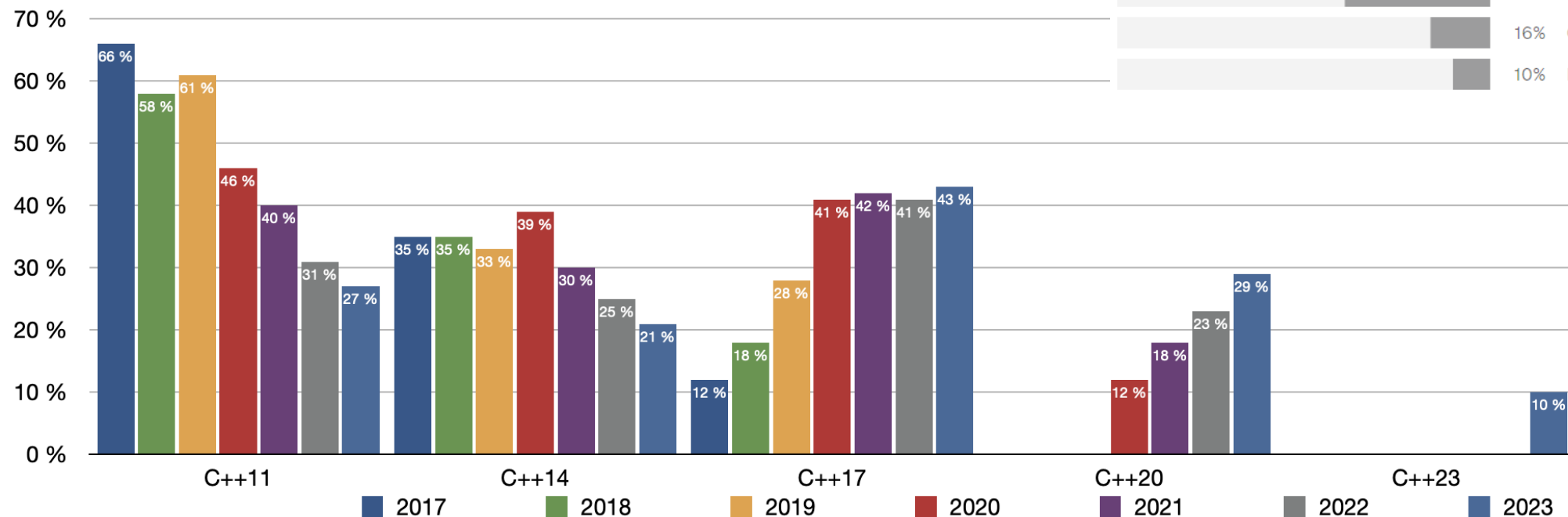
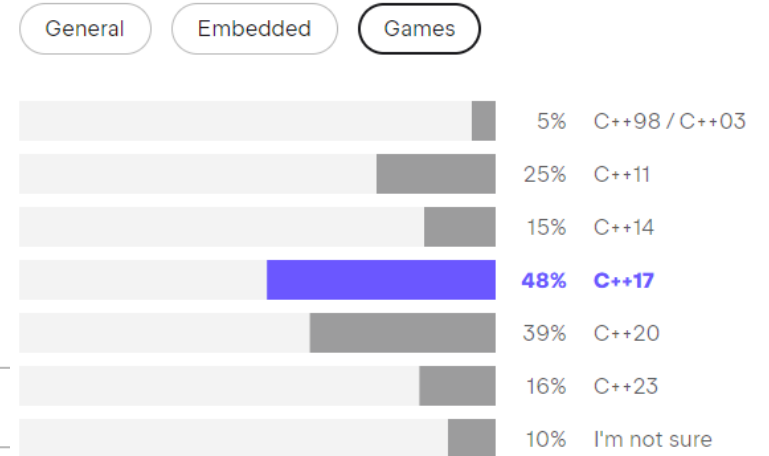
C++ Ecosystem in 2023

<https://blog.jetbrains.com/clion/2024/01/the-cpp-ecosystem-in-2023/>



■ Besonders **Spieleentwicklungs-Branche** setzt auf neuere C++ Standards

- Starker Anstieg mit C++26 absehbar, wenn Reflektionen (ein meist-erwartetes Feature) dem Sprachstandard hinzugefügt werden



C++: Eine kurze Einführung



Warum C++?

■ Zero-Cost Abstraktionen

- C++ erlaubt eigene Abstraktionen (Variablen, Funktionen, Klassen mit eigenen Namen und Strukturen) und bietet Built-In Möglichkeiten
- Neue C++ Versionen führen neue Abstraktionen ein (z.B. den range-based for-Loop)
- C++ Abstraktionen sind laufzeitoptimiert, im Bezug auf Speicher- und Zeitbedarf
- C++ gibt dem Programmierer immer viel Kontrolle und Macht über den Maschinencode, der letztendlich erzeugt wird, um für jeden Anwendungsfall die beste Performanz zu ermöglichen
- Die Performanz und Sicherheit von C++ Programmen liegt komplett bei dem Programmierer

■ Schreiben und Verwalten von großen, mächtigen und robusten Code-Basen

- C++ erlaubt betriebssystemnahe (low-level) Programmierung und bietet gleichzeitig effiziente Bibliotheken, um diese nicht selbst implementieren zu müssen

C++: Eine kurze Einführung



Warum C++? – C vs C++

- „Wie viele Kopien von Hamlet befinden sich unter den Büchern?“

// C version

```
struct string_elem_t {  
    const char* str_;  
    string_elem_t* next_;  
};  
  
int num_hamlet(string_elem_t* books) {  
    const char* hamlet = "Hamlet";  
    int n = 0;  
    string_elem_t* b;  
    for (b = books; b != 0; b = b->next_)  
        if (strcmp(b->str_, hamlet) == 0)  
            ++n;  
    return n;  
}
```

// C++ version

```
int num_hamlet(const std::forward_list<std::string>& books) {  
    return std::count(books.begin(), books.end(), "Hamlet");  
}
```

- C Pointer zu rohen Speicheradressen sind nicht mehr sichtbar
- Der `std::forward_list<std::string>` ersetzt die händisch erzeugte Linked-List `string_elem_t`
- Die `std::count()` Funktion ersetzt den `for`-Loop und die `if`-Abfrage
- Die `std::string` Klasse bietet eine höhere Abstraktion als `char*` und `strcmp()`

C++: Eine kurze Einführung

Warum C++?



■ Zero-Overhead Prinzip

- „Was Du nicht nutzt, kostet Dich auch nichts“
- „Was Du nutzt, könntest Du selbst nicht besser implementieren“
- C++ Abstraktionen besitzen keinen Performanz-Overhead
- Ausnahmen sind Exceptions und Run-Time Type Information (RTTI)
- C++ Abstraktionen sind primär auf Performanz ausgerichtet und machen den Code lesbarer und einfacher zu verwalten

■ Portabilität

- C++ ist keine Erweiterung für C, aber immer noch mit C kompatibel
- Konzepte von C++ lassen sich auf andere Programmiersprachen übertragen

■ Robustheit

- C++ bietet zahlreiche Funktionalitäten, um ausdrucksstarken und robusten Code zu schreiben
- Gleichzeitig kann C++ für fast jede Hardware und Echtzeitanforderung verwendet werden

C++: Eine kurze Einführung



Vergleich mit anderen Programmiersprachen

- Seit C++ entwickelt wurde, sind viele neue Typen, Plattformen und Programmiersprachen entstanden
 - C++ ist weiterhin eine weit verbreitete und genutzt Sprache mit Compilerunterstützung auf den meisten Plattformen
 - Große Ausnahme (bisher) ist die Web-Entwicklung, bei der JavaScript und ähnliche Technologien das Fundament bilden
 - Web-Anwendungen erhalten jedoch immer mehr Anforderungen und sollen immer mehr berechnen können, wodurch C++ den Einstieg in Web-Entwicklung mit Technologien wie Emscripten, asm.js und WebAssembly gefunden hat.
- Nachfolgend der Vergleich zwischen C++ und Java
 - ähnlich für alle Garbage-Collection basierten Programmiersprachen wie C# und JavaScript

C++: Eine kurze Einführung



Vergleich mit anderen Programmiersprachen

■ Kompilierung

- Java kompiliert erst in Bytecode, der erst zur Laufzeit in Maschinencode übersetzt wird
- C++ Implementierungen kompilieren den Quellcode direkt in Maschinencode
- Theoretisch kann beides zur selben Performanz führen, bisher ist der C++ Ansatz jedoch performanter

■ Freigabe von Speicherplatz

- Java gibt reservierten Speicherplatz automatisch über einen Garbage Collector
- In C++ muss reservierter Speicherplatz manuell oder durch Referenzzählung freigegeben werden
- Garbage Collectoren verhindern Memory Leaks, allerdings zu Kosten von Performanz und Vorhersehbarkeit

■ Allokationen

- Java speichert alle Objekte einzeln und separat im Heap
- C++ erlaubt dem Programmierer, Objekte auf dem Stack oder Heap zu speichern
- C++ ermöglicht, mehrere Objekte in einer einzelnen Heap-Allokation zu erzeugen

C++: Eine kurze Einführung



Vergleich mit anderen Programmiersprachen

■ Value Semantics

- C++ erlaubt es, Objekte als Referenz oder als Kopie (Value) zu übergeben
- Das C++ Typ System ermöglicht, explizit den Besitzer eines Objekts anzugeben
- Bei Java werden Objekte immer als Referenz übergeben

```
// Die Bagel teilen sich nicht dieselben Toppings
// Liste von Toppings
auto t = std::set<std::string>{};
t.insert("salt");
auto a = Bagel{t};
// 'a' ändert sich nicht, dadurch
// dass 't' Pfeffer hinzugefügt wird
t.insert("pepper");
// 'a' hat Salz
// 'b' hat Salz & Pfeffer
auto b = Bagel{t};
// Kein Bagel wird sich ändern
t.insert("oregano");
```

C++

```
// Die Bagel teilen sich die Toppings
// Liste von Toppings
TreeSet<String> t = new TreeSet<String>();
t.add("salt");
Bagel a = new Bagel(t);

// 'a' hat nun auch Pfeffer
t.add("pepper");

// 'a' und 'b' haben Salz & Pfeffer
Bagel b = new Bagel(t);
// Beide Bagel werden sich ändern
t.add("oregano");
```

Java

C++: Eine kurze Einführung

Vergleich mit anderen Programmiersprachen



■ Const Correctness

- C++ besitzt im Vergleich zu Java und vielen anderen Programmiersprachen **Konstanten-Korrektheit**
- Für Funktionen wird explizit angegeben, ob diese Objekt-Informationen ändern dürfen oder nicht

■ Object Ownership

- C++ ermöglicht die Umsetzung strikter Besitzrechte für Objekte, während bei Java alle Objekte geteilt werden können
- Dies verbessert das Memory-Management sowie die Nutzbarkeit und Robustheit des Codes

■ Deterministische Objektzerstörung

- Die Zerstörung von Objekten in C++ ist immer deterministisch
- Wir können immer genau wissen, wann ein Objekt zerstört wird
- Zum Ende der Lebenszeit eines Objekts kann alles zurückgesetzt werden, was während der Lebenszeit geschehen ist

C++: Eine kurze Einführung



Nachteile von C++

- C++ besitzt mehr Konzepte, die es zu verstehen und lernen gilt, als andere Programmiersprachen
 - C++ ist schwer zu meistern und korrekt zum vollen Potenzial zu nutzen
 - Ein guter C++ Programmierer nutzt die höhere Komplexität zum Vorteil, um besseren, performanteren und robusteren Code zu erzeugen
- C++ besitzt jedoch lange Kompilierzeiten und ein Komplexes System, um Bibliotheken zu importieren
 - Mit C++20 wurden allerdings Modules eingeführt, die das Importieren von Bibliotheken und die Kompilierzeit verbessern.
 - Modules beginnen sich jedoch erst noch zu verbreiten.
- C++ stellt (mehr oder weniger) nur grundsätzliche Bibliotheken zur Verfügung
 - Andere Sprachen besitzen bereits Bibliotheken für die meisten Anwendungen, wie Grafikanwendungen, Nutzerinterfaces, Netzwerke, Threading, Ressourcenverwaltung, ...
 - Bei C++ werden für größere/spezifischere Anwendungen externe Bibliotheken benötigt

Standard Bibliothek

Ausgewählte Header

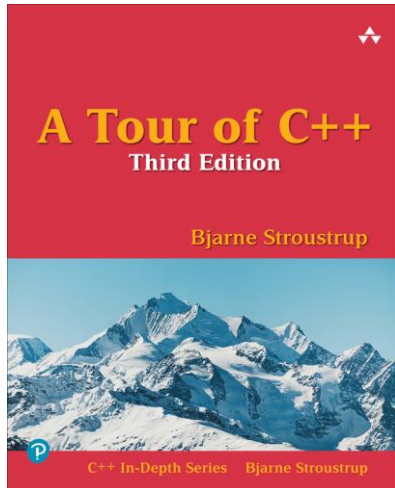


Header	Inhalte (Auszug)	Vorlesungsthema
<algorithm>	copy(), find(), sort()	Container
<array>	array	Container
<chrono>	duration, time_point, month	Übungen
<cmath>	sqrt(), pow()	
<complex>	complex, sqrt(), pow()	
<concepts>	floating_point, copyable, predicate	Templates
<filesystem>	path	Input & Output
<fstream>	fstream, ifstream, ofstream	Input & Output
<functional>	function, greater_equal, hash	Funktionale Prog.
<future>	future, promise	
<ios>	hex, dec, scientific, fixed	Input & Output
<iostream>	istream, ostream, cin, cout	Input & Output
<map>	map, multimap	Container
<memory>	unique_ptr, shared_ptr, allocator	Speicherverwaltung
<mutex>	mutex, lock_guard, unique_lock	Multithreading

Header	Inhalte (Auszug)	Vorlesungsthema
<random>	default_random_engine	Übungen
<ranges>	sized_range, subrange, take()	
<regex>	regex, smatch	Input & Output
<string>	string, basic_string	Container
<string_view>	string_view	
<set>	set, multiset	Container
<sstream>	istringstream, ostringstream	Input & Output
<stdexcept>	length_error, out_of_range	Fehlerbehandlung
<tuple>	tuple, get<>(), tuple_size<>	Container
<thread>	thread	Multithreading
<type_traits>	is_same, is_arithmetic	Templates
<unordered_map>	unordered_map	Container
<utility>	move(), swap(), pair	Speicherverwaltung
<variant>	variant	Fehlerbehandlung
<vector>	vector	Container

Literatur

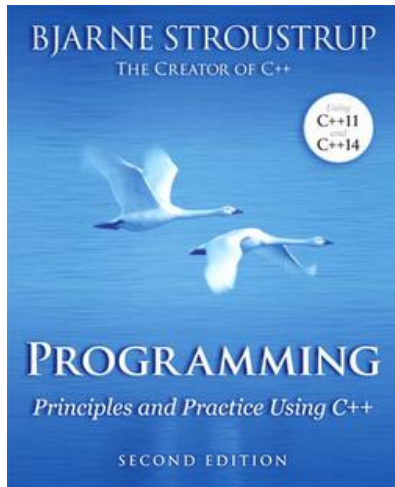
Buch-Quellen



Kompakter Überblick über die C++ Sprache
bis zum C++20 Standard:

A Tour of C++, Third Edition, Bjarne Stroustrup, 2022
Pearson Education Limited, ISBN 978-0-13-681648-5

Eine Tour durch C++, 3. Edition, Bjarne Stroustrup, 2023
Mitp Verlags GmbH & Co.KG, ISBN 978-3-7475-0625-7

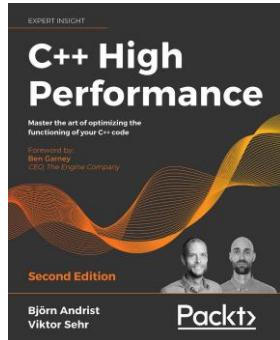


Ausführliches Einsteigerbuch für jeden anstrebenden C++
Programmierer, mit zahlreichen Beispielen und Übungen:

Programming: Principles and Practice Using C++, Second Edition,
Bjarne Stroustrup, 2014,
Pearson Education Limited, ISBN 978-0-321-99278-9

Literatur

Buch-Quellen

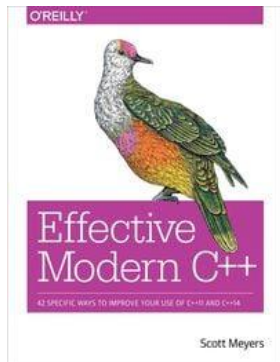
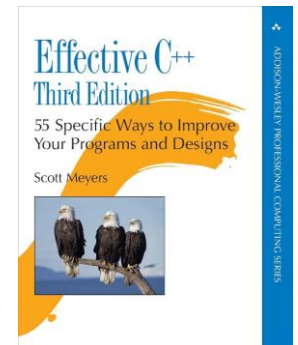


Weiterführende Inhalte, Techniken und Tipps:

C++ High Performance: Master the art of optimizing the functioning of your C++ code, Second Edition, Björn Andrist, Viktor Sehr, 2020
Packt Publishing, ISBN 978-1839216541

Tipps und Tricks für effizientes Programmieren mit C++:

Effective C++, Third Edition, Scott Meyers, 2005
Addison-Wesley Professional, ISBN 978-0-32-133487-9

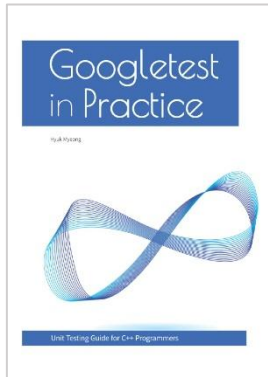


Effizientes Programmieren für C++11 und C++14:

Effective Modern C++, Scott Meyers, 2015
O'Reilly Media, ISBN 978-1-4919-0399-5

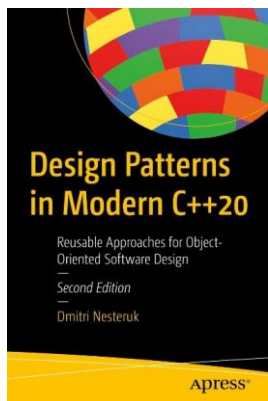
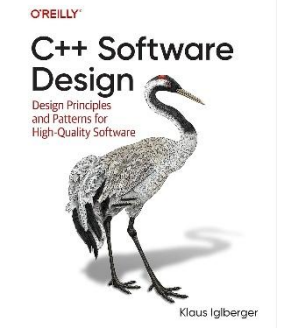
Literatur

Buch-Quellen



Unit Testing Guide für C++ Programmierer:
Googletest in Practice, Hyuk Myeong, 2021
Independently published, ISBN 979-8-7736-7866-3

Software Design Prinzipie und Patterns für C++:
C++ Software Design: Design Principles and Patterns for High-Quality Software, Klaus Iglberger, 2022
O'Reilly Media, ISBN 978-1-0981-1316-2



Software/Design Patterns für C++ bis C++20:
Design Patterns in Modern C++20, Dmitri Nesteruk, 2022
Apress Berkeley, ISBN 978-1-4842-7294-7
(verfügbar über den Springer Link)

Literatur

Online-Quellen



Dokumentation der Funktionalitäten von C++, insbesondere der Standardbibliothek, mit Beispielen zur Verwendung:

C++ Reference: <https://en.cppreference.com/>

Ausführliche Erklärungen zu C++ Mechaniken im FAQ-Stil mit Tipps zur Verwendung und Beispielen für unterschiedliche Anwendungsfälle:

ISO-C++: <https://isocpp.org/faq>